Prueba Técnica — Ingeniero de Datos Junior

Versión 1.0 — Carga incremental del padrón electoral

1. Objetivo

Queremos ver cómo **programas y cómo resuelves problemas** con datos reales y sucios. En esta prueba **tomarás el archivo diario del padrón electoral, lo limpiarás y cargarás a una base PostgreSQL sin duplicar información.**

Prepara el entorno siguiendo el README (instalar Python y Docker o PostgreSQL, y crear la base).

2. Contexto del caso

Se está desarrollando un dashboard de resultados electorales de un país. Tú eres el ingeniero de datos del equipo y eres responsable de que el **padrón de empadronados** llegue a la base de datos de forma confiable y actualizada.

La base de datos ya existe y está en producción: **trabajas sobre ella tal como está** (`sql/01_ddl_electoral.sql`; el diccionario de datos está en `docs/BASE_DE_DATOS.md`). La tabla que vas a cargar es `electoral.empadronados`.

2.1 La fuente

`data/empadronados/empadronados_YYYYMMDD.csv`: llega un archivo por día. Se entregan tres días consecutivos: `20260302`, `20260303` y `20260304`.

Cada archivo es una fotografía completa del padrón a esa fecha, no solo los cambios del día. Entre un día y el siguiente hay:

- **altas**: empadronados nuevos que no existían el día anterior,

- **cambios**: empadronados que cambiaron de centro y mesa (traslados),
- **bajas**: empadronados que pasaron a estado `suspendido` o `fallecido`. Una
  baja **no** borra al ciudadano: solo cambia su estado.

2.2 Los datos vienen sucios

Los archivos reflejan la realidad y traen inconsistencias. No están documentadas una por una a propósito: encontrarlas es parte de la prueba. Para que no pierdas tiempo buscando a ciegas, mira con atención estas columnas: `sexo`, `rango_edad`, `estado_registro` y `fecha_inscripcion`, y revisa también si hay **filas repetidas**.

Lo que encuentres anótalo en `DECISIONES.md`, con un ejemplo real y qué decidiste hacer.

2.3 Forma estándar que debe tener cada columna

Para que no tengas que adivinar, estas son las formas finales que queremos en la base de datos:

Columna	Forma estándar
`sexo`	`M` o `F`. Si viene vacío, se guarda como nulo (`NULL`)
`estado_registro`	`activo`, `suspendido` o `fallecido` (en minúsculas)
`rango_edad`	`18-25`, `26-35`, `36-45`, `46-60` o `61+`
`fecha_inscripcion`	`YYYY-MM-DD`

3. Qué debes desarrollar

El esqueleto en `src/` ya trae lista la lectura del CSV, la conexión a la base y el flujo `extraer → transformar → validar → cargar`. Lo que falta está marcado con `TODO`. Empieza por `src/pipelines/empadronados_pipeline.py` para ver cómo encajan las piezas.

Parte 1 — Transformación de datos

Implementa en `src/transformers/common.py`:

- `StandardizeCategories`: deja `sexo`, `estado_registro` y `rango_edad` en la forma estándar de la sección 2.3.
- `ParseDates`: la fecha de inscripción viene en dos formatos distintos; déjala siempre como `YYYY-MM-DD`.
- `Deduplicate`: elimina las filas repetidas por `id_empadronado`.

Luego conecta tus transformaciones en `empadronados_pipeline.py`. Piensa en qué **orden** deben ejecutarse y por qué.

Si llegas con tiempo: en `src/quality/validations.py` hay tres reglas sencillas para implementar. Un registro que no cumple una regla no debe
desaparecer en silencio: descártalo, cuéntalo y deja un mensaje en el log con el motivo.

Listo cuando: puedes mostrar, con datos reales de un archivo, un antes y un después de cada transformación. No se exige `pytest` ni pruebas automatizadas (si quieres escribirlas, es opcional): sirve un script corto, un `print` o la salida del propio flujo.

Parte 2 — Carga incremental del padrón

`python main.py load-empadronados --fecha 2026-03-03` debe procesar el archivo de esa fecha. Implementa `IncrementalLoader` en `src/loaders/postgres_loader.py` para que:

1. **Maneje altas, cambios y bajas**: un empadronado nuevo se inserta; uno que ya existe se actualiza con los datos nuevos (traslado o cambio de estado).
2. **No duplique**: correr la misma fecha dos veces deja la tabla con las mismas filas.
3. **Registre hasta qué fecha se procesó** (marca de agua). Puedes escribir una fila en `electoral.carga_log`.
4. **Informe cuántas filas cargó**: el flujo ya muestra al terminar las filas leídas y las cargadas; asegúrate de que `load` devuelva el número correcto.

Listo cuando: cargas `02 → 03 → 04` en orden, vuelves a correr el `03`, y
con consultas SQL puedes demostrar que no hay duplicados y explicar el estado en que quedó la tabla.

Para sumar puntos extra (si sobra tiempo):

- Informar cuántas altas, cuántos cambios y cuántas bajas detectó cada carga.
- Evitar que un archivo viejo pise a uno más nuevo (por ejemplo, reprocesar el `03` cuando ya se cargó el `04`).

Parte 3 — Análisis de la base de datos (opcional)

Si te sobra tiempo, anota en la sección 5 de `DECISIONES.md` entre 3 y 5
problemas del modelo de datos que notaste al cargar el padrón: qué consecuencia tiene cada uno y una mejora posible. Una línea por cada punto; no se pide un diagrama ni un modelo nuevo.

Reglas

- **No modifiques** las tablas de `sql/01_ddl_electoral.sql`.
- Las credenciales van **solo** en variables de entorno (`.env`), nunca en el código.
- Código que otra persona pueda leer: nombres claros y funciones cortas.
- Puedes agregar librerías si lo necesitas; anota cuál y por qué en
  `DECISIONES.md`.

4. Entregables

Un repositorio de GitHub (público) con:

1. El código del ETL (`src/`).

2. `DECISIONES.md` completado: hallazgos en los datos, tus decisiones, y qué quedó pendiente.
3. `README.md` con la sección “Cómo ejecutar mi solución” completada: los comandos para reproducir tu trabajo, en orden.

Git: haz **al menos 3 commits** con mensajes descriptivos (no un único commit al final). No subas el archivo `.env`. Como extra, trabaja en una rama `feature/etl` y abre un Pull Request hacia `main` (déjalo abierto).

5. Uso de asistentes de IA y sustentación

Se permite usar asistentes de IA y no lo penalizamos. A cambio, **tienes que entender y poder defender todo lo que entregas.** Después de la entrega habrá una sustentación corta en vivo donde nos explicas cómo funciona tu solución y por qué tomaste tus decisiones.

6. Si el tiempo no alcanza

Prioriza en este orden y deja constancia en `DECISIONES.md` de lo que quedó pendiente y cómo lo harías:

1. Parte 1: transformaciones funcionando.
2. Parte 2: carga sin duplicados (puntos 1 y 2).
3. Marca de agua e informe de filas (puntos 3 y 4).
4. `DECISIONES.md` y `README.md`.
5. Extras de la Parte 2, validaciones y Parte 3.